

Manual Ampliado de Python para PySpark

1. Introducción

PySpark es el API de Python para Apache Spark. Permite procesar grandes volúmenes de datos distribuidos usando Python. Este manual incluye ejemplos avanzados para análisis de datos.

2. Recordatorio de Python Básico

Variables, listas, diccionarios, condicionales, bucles y funciones funcionan igual que en el manual básico.

3. Pandas

Pandas es una librería de Python para manipulación y análisis de datos. Permite trabajar con tablas de datos (como hojas de Excel o CSV) de forma muy parecida a Excel, pero en código. Sus estructuras principales son:

- > DataFrame → tabla bidimensional (filas y columnas).
- > Series → una columna de datos.

Es ampliamente usada en data science, Big Data y machine learning.

- Exportar un Excel a .csv

```
import pandas as pd

# Leer Excel
df = pd.read_excel("clientes.xlsx", sheet_name="Clientes")

# Guardar CSV
df.to_csv("clientes.csv", index=False)
```

- Crear un Dataframe con Pandas

```
import pandas as pd

data = [
    {"nombre": "Ana", "edad": 25, "ciudad": "Madrid"},
    {"nombre": "Luis", "edad": 35, "ciudad": "Barcelona"}
]

# Crear DataFrame
df = pd.DataFrame(data)

print(df)

# Filtrar por edad > 30, agrupar por ciudad y contar
df_mayores_30 = df[df["edad"] > 30]

print(df_mayores_30)
```

4. SparkSession y DataFrames

```
from pyspark.sql import SparkSession
spark = SparkSession.builder.appName("EjemploAvanzado").getOrCreate()

.....

spark.close
```

- Crear DataFrame desde lista de diccionarios

```
data = [
    {"nombre": "Ana", "edad": 25, "ciudad": "Madrid"},
    {"nombre": "Luis", "edad": 35, "ciudad": "Barcelona"},
    {"nombre": "Pedro", "edad": 40, "ciudad": "Madrid"},
    {"nombre": "Laura", "edad": 28, "ciudad": "Valencia"}
]

df = spark.createDataFrame(data)
df.show()
```

- Leer CSV con inferencia de tipos

```
df_csv = spark.read.csv("datos.csv", header=True, inferSchema=True)
df_csv.show()
```

5. Operaciones Avanzadas con DataFrames

- Selección y filtrado

```
df.select("nombre", "edad").filter(df["edad"] > 30).show()
```

- Agregaciones

```
from pyspark.sql.functions import avg, max, min
```

```
df.groupBy("ciudad").agg(  
    avg("edad").alias("edad_media"),  
    max("edad").alias("edad_maxima"),  
    min("edad").alias("edad_minima")  
) .show()
```

- Crear columnas nuevas

```
from pyspark.sql.functions import col
```

```
df = df.withColumn("edad_doble", col("edad") * 2)  
df.show()
```

- Ordenar datos

```
df.orderBy(col("edad").desc()).show()
```

- Joins entre DataFrames

```
otros_datos = [  
    {"ciudad": "Madrid", "poblacion": 3200000},  
    {"ciudad": "Barcelona", "poblacion": 1600000}  
]  
df_ciudades = spark.createDataFrame(otros_datos)  
  
df.join(df_ciudades, on="ciudad", how="left").show()
```

6. RDDs y Funciones Lambda

Aquí se crea un RDD a partir de una Lista Python [1, 2, 3, 4, 5]. parallelize reparte los datos entre varias particiones en Spark.

```
rdd = spark.sparkContext.parallelize([1, 2, 3, 4, 5])
```

*# map() aplica una función a cada elemento del RDD. Aquí se usa una función lambda (lambda x: x*2) que simplemente multiplica cada valor por 2.*

```
rdd2 = rdd.map(lambda x: x*2)
```

collect() trae todos los elementos del RDD al driver (tu programa Python). Por eso ves [2, 4, 6, 8, 10]. Esto es la base de la programación funcional en Spark: usar operaciones como map, filter, reduce, etc., sobre RDDs.

```
print(rdd2.collect()) # [2, 4, 6, 8, 10]
```

7. Guardar Resultados

Guardar en CSV

```
df.write.csv("resultado.csv", header=True)
```

Guardar en Parquet

```
df.write.parquet("resultado.parquet")
```

8. Tips y Buenas Prácticas

1. Evitar loops sobre DataFrames grandes: usar transformaciones de Spark.

- No es eficiente recorrer un DataFrame fila por fila con `for` en Python.
- En lugar de eso, usa funciones nativas de Spark (`withColumn`, `select`, `filter`, `groupBy`, etc.), que están optimizadas para ejecutarse en paralelo en el clúster.

Ejemplo malo:

```
for row in df.collect():  
    # Procesar en Python (lento)
```

Ejemplo bueno:

```
df = df.withColumn("nueva_col", df["col1"] + df["col2"])
```

- ### 2. Usar `persist()` o `cache()` para DataFrames que se reutilizan muchas veces.
- ### 3. Cuando aplicas varias transformaciones sobre un DataFrame, Spark lo recalcula desde cero cada vez que ejecutas una acción (`show()`, `count()`, `write()`, etc.).
- Si planeas usar el mismo DataFrame muchas veces, esto es **ineficiente**.
 - Para evitar recalcular, puedes **guardar el DataFrame en memoria o en disco** usando:
 - `cache()` → guarda en memoria (RAM)
 - `persist()` → permite elegir almacenamiento en memoria, disco o ambos (StorageLevel)

```
from pyspark.sql import SparkSession

spark = SparkSession.builder.getOrCreate()

df = spark.read.csv("datos.csv", header=True, inferSchema=True)

# Transformación
df_filtrado = df.filter(df["edad"] > 18)

# Guardamos en memoria para reutilizar varias veces
df_filtrado_cached = df_filtrado.cache()

# Primera acción: Spark ejecuta y guarda en memoria
df_filtrado_cached.show()

# Segunda acción: Spark reutiliza los datos ya cacheados
df_filtrado_cached.count()
```

- La primera acción ejecuta el plan completo y guarda el resultado en memoria.
- La segunda acción **no recalcula** todo desde cero, sino que usa los datos ya cacheados.

Diferencia entre `cache()` y `persist()`

- `cache()` → es equivalente a `persist(StorageLevel.MEMORY_ONLY)`
- `persist()` → más flexible, por ejemplo:

```
from pyspark import StorageLevel

df.persist(StorageLevel.MEMORY_AND_DISK)
```

- Esto guarda los datos en memoria, y si no caben, en disco.

4. Siempre especificar `header=True` y `inferSchema=True` al leer CSVs.

- Cuando lees un CSV en PySpark, por defecto:
 - **No hay cabecera:** todas las columnas se llaman `_c0`, `_c1`, `_c2`, etc.
 - **Todos los datos se leen como strings:** incluso si hay números o fechas.
- Por eso, es recomendable usar:

```
header=True      # La primera fila del CSV se usa como nombres de columnas
inferSchema=True # Spark infiere automáticamente el tipo de datos de cada
```

Ejemplo:

```
from pyspark.sql import SparkSession

spark = SparkSession.builder.getOrCreate()

# Leer CSV con header y tipos inferidos
df = spark.read.csv("datos.csv", header=True, inferSchema=True)

# Mostrar esquema y primeras filas
df.printSchema()
df.show(5)
```

- `printSchema()` mostrará los tipos correctos (`IntegerType`, `StringType`, etc.), no solo `StringType`.
- `show()` mostrará los datos con los nombres de columnas correctos.

5. Revisar la documentación de funciones en `pyspark.sql.functions` para operaciones avanzadas.
6. PySpark es perezoso: las transformaciones no se ejecutan hasta que se llama `.show()`, `.collect()` o `.write()`.
 - Cuando haces transformaciones sobre un DataFrame (`select`, `filter`, `withColumn`, `groupBy`, etc.), **Spark no ejecuta nada inmediatamente**.
 - En vez de procesar al instante, Spark construye un **plan lógico** con todas las operaciones pendientes. Ese plan se ejecuta en cuando llamas a una **acción** como:
 - `.show()` → para ver filas en pantalla.
 - `.collect()` → para traer los datos completos al driver.
 - `.count()` → para contar filas.
 - `.write()` → para guardar los resultados en disco (CSV, Parquet, etc.).

Ejemplo:

```
# Defino transformaciones, pero Spark no ejecuta todavía
df2 = df.filter(df["edad"] > 18).select("nombre", "edad")

# Aquí Spark todavía no ha leído ni procesado los datos
print("Transformaciones definidas, pero nada ejecutado")

# Ahora sí se ejecuta el plan, porque llamamos a una acción
df2.show()
```

El `.filter` y el `.select` no corrieron cuando los escribiste, sino en cuanto se usa `.show()`.

Ventaja: Spark puede optimizar todo el flujo de operaciones antes de ejecutarlas (ejemplo: combinar filtros, mover operaciones más pesadas al final, etc.).